

Performance of Prophet in Stock-Price Forecasting: Comparison with ARIMA and MLP Networks

Özüm Emre Aşırım
TUM School of Computation, Information
and Technology
Technical University of Munich
Garching, Germany
ozum.asirim@tum.de

Adil Aşırım
Department of Management
Istanbul Aydın University
Istanbul, Turkey
adilasirim@stu.aydin.edu.tr

Murat Adil Salepcioglu
Department of Management
Istanbul Aydın University
Istanbul, Turkey
muratsalepcioglu@aydin.edu.tr

Abstract— Prophet, an open-source forecasting tool developed by Facebook, is increasingly being used to predict stock prices. This study compares Prophet with two other popular forecasting tools: ARIMA and Multi-layer Perceptron (MLP) networks. Prophet is particularly precise at identifying key turning points and adjusting for holidays and outliers, making it suitable for capturing long-term trends. On the other hand, ARIMA is better suited for short-term predictions, especially in cases with stable, linear trends. Meanwhile, MLP networks excel in managing complex datasets and capturing strong nonlinear patterns. In our analysis, we used data from Turkey's BIST 30 index and a selection of 100 corporate stocks from Yahoo Finance. We evaluated the accuracy of each method by comparing the predicted stock values against actual outcomes using correlation coefficients. The goal of this study is to determine how well Prophet performs in forecasting stock prices compared to these two commonly used methods.

Keywords—Prophet, ARIMA, Time-series forecasting, Deep learning, Stock-market

I. INTRODUCTION

Prophet, developed by Taylor and Letham in 2017 at Facebook, tackles the complexities often found in time series data, which many traditional forecasting tools fail to handle effectively. This includes challenges like missing values, significant outliers, and abrupt changes in seasonality—issues that are especially prevalent in areas such as stock market forecasting. Prophet is an accurate forecasting tool because it is built around a decomposable model that separates the data into three key components: trend, seasonality, and holidays. This approach is different from standard models that often impose strict seasonal patterns. Instead, Prophet introduces a more flexible way to capture patterns at different timescales—yearly, weekly, and daily—also incorporating holiday effects. It achieves this by employing a Bayesian curve fitting method to adjust nonlinear trends that handle these variations.

Literature shows how Prophet is not only useful in traditional areas like product demand and web traffic forecasting, but also in retail, capacity planning, and public health. Comparative analysis by Taylor and Letham highlights Prophet's strengths in managing data with strong seasonal variations and gaps. Additionally, a study by Zhang et al. (2019) explores Prophet's

effectiveness with large-scale datasets, noting its scalability and robustness, which outperforms many ARIMA-based models and machine learning models. This versatility of Prophet is particularly valuable in analyzing the unpredictable stock market, inviting further exploration and comparison with both traditional and modern forecasting techniques to better understand its advantages and limitations in financial analytics.

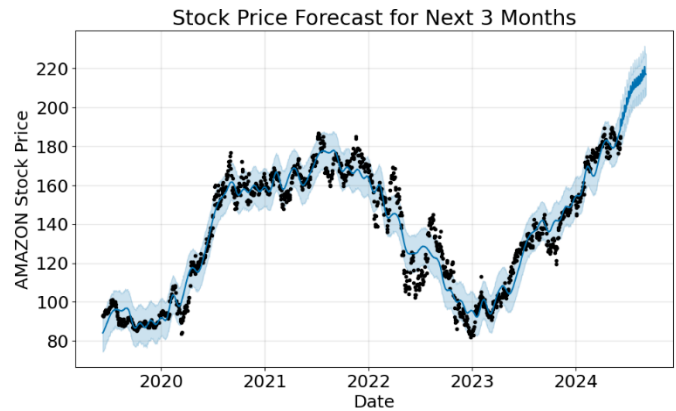


Fig. 1. Forecasting AMAZON stock price for the next 3 months using Prophet. Figure includes the training data (black), fitted data, and the forecasted data.

In financial analytics, getting stock price forecasts right is crucial for informed decision-making (see Figure 1). Recently, among the various tools available, Prophet, ARIMA, and Multilayer Perceptrons (MLPs) are particularly noteworthy for their strengths and popularity. This paper takes a closer look at Prophet, a relatively new tool, known for its robust approach to managing seasonal fluctuations and outliers. What might set Prophet apart in the field of economic forecasting is its effectiveness, user-friendliness, and its ability to handle the often irregular data patterns found in stock market datasets.

Exploring the relative merits of forecasting tools such as Prophet in comparison to traditional tools like ARIMA, is important for appropriate model selection. While ARIMA has been revered for its accuracy in capturing the linear trends that pervade short-term financial analyses [1,5,6], it often fails when confronted with non-linear dynamics and pronounced seasonal fluctuations—challenges where Prophet tends to excel

[2,4,5,6,8,10]. Additionally, the growing use of Multilayer Perceptrons (MLPs) in predicting stock prices, which demonstrates the recent application of deep learning techniques in financial analysis [7,8,9], necessitates an inclusion in this comparison. Known for their ability to handle complex, non-linear patterns in large datasets, MLPs are an important standard for evaluating Prophet's effectiveness [8]. This analysis aims to determine whether Prophet's simpler, less resource-intensive methods can match or even surpass the performance of these complex neural networks in financial forecasting tasks.

In this study, we evaluate the performance of the aforementioned forecasting algorithms through a method known as correlation-coefficient-based backtesting. This process involves generating predictions for specified time windows, and comparing these predictions against actual observed values. We then calculate a correlation coefficient for each comparison, which quantifies the linear relationship between the predicted and actual values.

To conduct a comprehensive analysis, we employ a method where the datasets for training and testing the models are incrementally shifted backward. This approach enables us to carry out multiple evaluations across different temporal spans, effectively capturing diverse market conditions and trends. Each shift produces a new set of predicted versus actual values, from which we derive a correlation coefficient. These coefficients, accumulated from numerous backtests, offer a reliable measure of each model's predictive power. Finally, we compile these coefficients into a histogram to visually depict the overall performance of the forecasting algorithms.

In this paper, we aim to compare forecasting models in two markedly different market contexts to evaluate their performance under different conditions. Our first focus is on the Turkish BIST 30 market, which consists of Turkey's premier 30 stocks. Noted for its more quantifiable and well-defined behavior, this market often presents clear trends over the long term, despite its short-term fluctuations [10]. On the other hand, we explore a more unpredictable dataset comprising daily stock prices from 100 varied companies, gathered randomly using the yfinance library in Python from Yahoo Finance. This dataset, with its inherent variability, presents a stricter test of the models' robustness. For both datasets, covering April 2012 to April 2024, our analysis aims to determine how well the forecasting models can adapt to both predictable and turbulent market scenarios.

II. METHODS

A. Correlation-coefficient based backtesting

Correlation-coefficient based backtesting is a method used to evaluate the accuracy of forecasting models by measuring the degree to which the predicted values from a model correlate with the actual historical values. The correlation coefficient is a statistical measure that calculates the strength and direction of a linear relationship between two variables. It ranges from -1 to 1, where 1 indicates a perfect positive linear relationship, -1 indicates a perfect negative linear relationship, and 0 indicates no linear relationship. In the context of backtesting forecasting models like Prophet, ARIMA, and MLP networks, a high

positive correlation coefficient close to 1 suggests that the model predictions closely match the actual data trends.

Threshold Value for Average Backtesting Correlation Coefficient in Accurate Predictions

While the exact threshold for what constitutes an "accurate" prediction can vary depending on the specific application and practical standards, a correlation coefficient of 0.8 or higher is typically considered indicative of a strong predictive model.

Interpreting Correlation Coefficients

- $r=1$: Perfect positive correlation. Predicted values exactly match the actual values.
- $r=0.9$ to $r=1$: Very strong positive correlation. Predictions are very close to actual values.
- $r=0.7$ to $r=0.9$: Strong positive correlation. Predictions are highly consistent with actual values.
- $r=0.5$ to $r=0.7$: Moderate positive correlation. Predictions are reasonably close to actual values but may have some significant deviations.
- $r<0.5$: Weak correlation. Predictions do not reliably follow actual values.

For forecasting models used in critical applications such as stock-price prediction, a higher threshold is generally expected. Considerations are given as follows:

1. *Financial Markets*: In the highly dynamic and unpredictable nature of financial markets [17-19], a correlation coefficient of $r \geq 0.7$ is often considered the minimum threshold for a model to be deemed reliable. Higher values closer to 0.9 or above are preferable if strong predictive power is desired [20].
2. *Business Forecasting*: In contexts like sales or demand forecasting, where the cost of prediction errors can be significant but not catastrophic, a threshold of $r \geq 0.5$ might be acceptable [20].

B. Implementation in Python

This section outlines the key Python implementations for evaluating the forecasting performance of Prophet, ARIMA, and MLP networks. Each model's implementation is demonstrated through essential code snippets that handle model fitting and prediction.

1. Prophet Implementation:

Using the Prophet library, the model can be set up and predictions can be made based on the following script:

```
from prophet import Prophet

model=Prophet(daily_seasonality=False,
yearly_seasonality=True)

model.fit(train_data)

future = model.make_future_dataframe(periods=90)

forecast = model.predict(future)
```

Correlation coefficients can then be calculated between the forecasts and actual data through backtesting (described below).

2. ARIMA Implementation: The *autoarima* function from *pmdarima* automatically identifies the best model parameters as follows:

```
from pmdarima import auto_arima

model=autoarima(train_data, seasonal=False,
suppress_warnings=True)

forecast = model.predict(n_periods=90)
```

3. MLP Network Implementation: An MLP network consists of an input layer IN, followed by N hidden layers [H1,H2,...,HN], and an output layer OUT, which we denote here as [IN,H1,H2,...,HN,OUT]. The MLP model is constructed here using TensorFlow's Keras API with sequential layers following the [IN,H1,H2,OUT]=[128,64,32,1] configuration, based on the Adam optimizer and the RELU activation function, as given in the following code snippet:

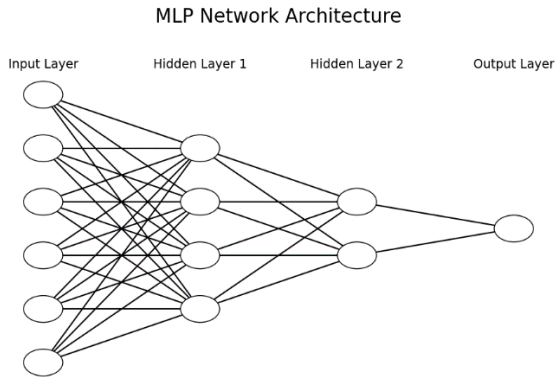


Fig. 2. An example MLP network architecture with two hidden layers following the [IN,H1,H2,OUT]=[6,4,2,1] configuration.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

model = Sequential([
    Dense(128, input_dim=10, activation='relu'), # look_back = 10
    days
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)])

model.compile(optimizer=Adam(learning_rate=0.001),
loss='mean_squared_error')

model.fit(X_train, Y_train, epochs=50, batch_size=16, verbose=0)

predictions = model.predict(X_test)
```

Following the implementation of all three models, correlations between predicted values and actual market data are computed to assess accuracy via backtesting. This is implemented as follows for the case of Prophet (which can also be easily adapted to ARIMA or MLP networks),

```
for i in range(n_periods):

    train_end = df['ds'].max() - pd.Timedelta(days=(n_periods - i)
* period_length)

    train_data = df[df['ds'] <= train_end]

    test_data = df[(df['ds'] > train_end) & (df['ds'] <= train_end
+ pd.Timedelta(days=period_length))]

    model = Prophet(daily_seasonality=False,
yearly_seasonality=True)

    model.fit(train_data)

    future =
model.make_future_dataframe(periods=period_length, freq='D')

    forecast = model.predict(future)

    forecasted_values = forecast[['ds',
'yhat']].set_index('ds').reindex(test_data['ds'])

    actual_values = test_data.set_index('ds')['y']

    valid_idx =
actual_values.dropna().index.intersection(forecasted_values.dropna()
.index)

    correlation = np.corrcoef(forecasted_values.loc[valid_idx,
'yhat'], actual_values.loc[valid_idx])[0, 1]

    correlations.append(correlation)
```

Based on these implementations, the corresponding findings are reported in the next section.

III. RESULTS

A. Backtesting for Prophet

Initially, backtesting was performed in the Turkish BIST30 market for Prophet. A total of 10 backtests, each with a duration of 1 month, 3 months, or 6 months, are performed per stock. For each stock (there are a total of 30 stocks in BIST 30), we used the training data from April 2012 plus [(backtest or iteration number)-1] × test duration, until April 2024 minus [10+1-(backtest or iteration number)] × test duration. Figures 1-3 show the resulting histograms corresponding to test durations of 1 month, 3 months, and 6 months respectively. Evidently, Prophet demonstrated some accuracy in predicting stock prices in the BIST30 market as observed by the skewed behavior of the histograms towards the right where correlation coefficient has greater values. Despite the significant presence of negative correlation values, these histograms show that it is reasonable to use Prophet for prediction in the Turkish BIST30 market.

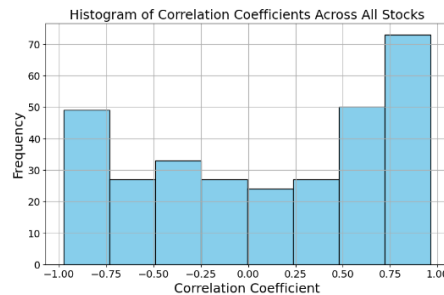


Fig. 3. Performance backtesting of Prophet based on a testing data window of 1 month in the Turkish BIST30 stock market.

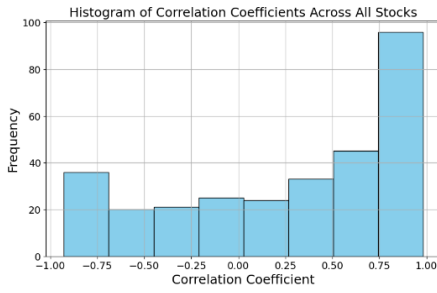


Fig. 4. Performance backtesting of Prophet based on a testing data window of 3 months in the Turkish BIST30 stock market.

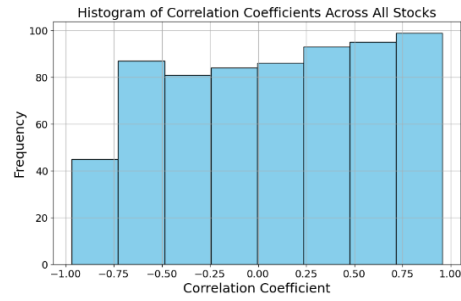


Fig. 7. Performance backtesting of Prophet based on a testing data window of 3 months for 100 randomly selected corporate stock data from Yahoo Finance.

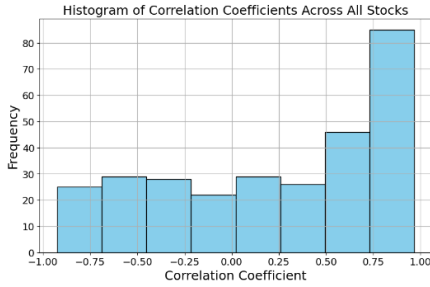


Fig. 5. Performance backtesting of Prophet based on a testing data window of 6 months in the Turkish BIST30 stock market.

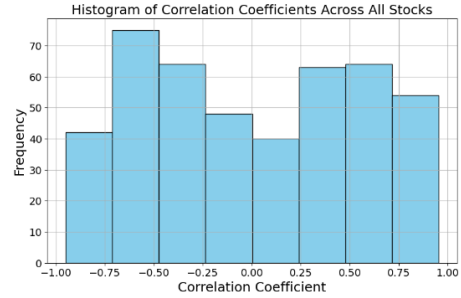


Fig. 8. Performance backtesting of Prophet based on a testing data window of 6 months for 100 randomly selected corporate stock data from Yahoo Finance.

In contrast, Figures 4-6 show the histograms corresponding to test durations of 1 month, 3 months, and 6 months for prediction in an arbitrary market as represented by the 100 randomly selected company stock lists (data from April 2012 to April 2024) from Yahoo Finance. Interestingly, Prophet did not show the same performance in this case. The ratio of correlation coefficients that have near-zero or negative values increased considerably. Moreover, there is a notable decrease in the prediction accuracy as the size of the test duration increases, as observed in Figures 4-6. This indicates that the use of Prophet, although providing some degree of accuracy (especially for shorter test durations), yields questionable predictions under the presence of greater uncertainty and diversity. Importantly, for a test duration of 6 months, the number of negative correlation coefficients exceeded the positive ones in the histogram.

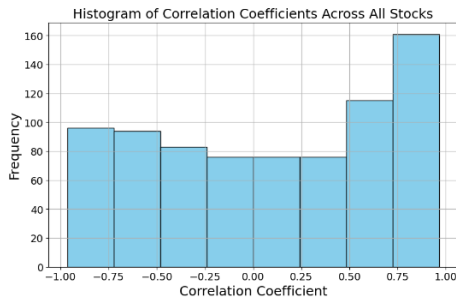


Fig. 6. Performance backtesting of Prophet based on a testing data window of 1 month for 100 randomly selected corporate stock data from Yahoo Finance.

B. Backtesting for ARIMA

The same backtests with identical training and test durations, are repeated for ARIMA. As before, initially the tests were performed in the BIST30 market. Despite indicating poorer performance than Prophet, ARIMA performed well in the BIST30 market, with the number of positive correlation coefficients in the generated histograms being greater than the negative ones for all test durations as indicated in Figures 7-9. This proved the efficacy of ARIMA in the BIST30 market, just as in the case of Prophet. Notably, some of the tests with ARIMA failed to measure a correlation coefficient (returned a NaN value) due to the intrinsic features of the *pmdarima* library of Python, but fortunately the number of cases where a correlation coefficient could be measured were enough to make a performance evaluation for ARIMA.

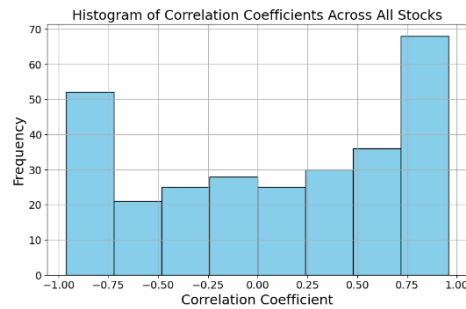


Fig. 9. Performance backtesting of ARIMA based on a testing data window of 1 month in the Turkish BIST30 stock market.

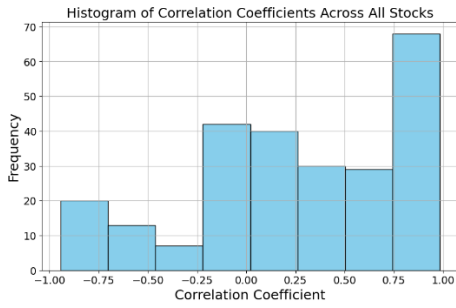


Fig. 10. Performance backtesting of ARIMA based on a testing data window of 3 months in the Turkish BIST30 stock market.

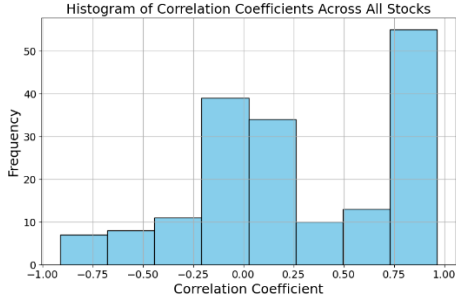


Fig. 11. Performance backtesting of ARIMA based on a testing data window of 6 months in the Turkish BIST30 stock market.

Expectedly, ARIMA also performed poorly in an arbitrary market as represented by the 100 randomly selected company stock lists (data from April 2012 to April 2024) from Yahoo Finance. Here, once again the performance is weaker than Prophet, with the number of negative correlation coefficients outnumbering positive ones for test durations of 3 and 6 months as indicated by Figures 10-12.

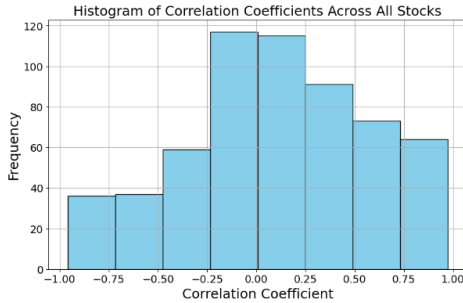


Fig. 12. Performance backtesting of ARIMA based on a testing data window of 1 month for 100 randomly selected corporate stock data from Yahoo Finance.

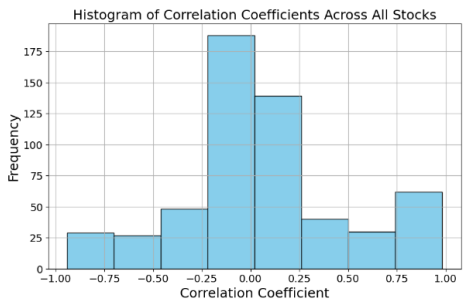


Fig. 13. Performance backtesting of ARIMA based on a testing data window of 3 months for 100 randomly selected corporate stock data from Yahoo Finance.

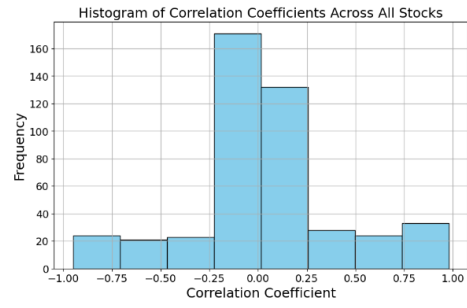


Fig. 14. Performance backtesting of ARIMA based on a testing data window of 6 months for 100 randomly selected corporate stock data from Yahoo Finance.

C. Backtesting for MLP networks

Finally, the backtests are performed for MLP networks. In our implementation (as previously discussed), the neural network contains 3 layers with 128, 64, and 32 neurons. The resulting correlation-coefficient histograms are only given for the randomly extracted list of stocks from Yahoo Finance as the results are very similar to BIST30 in this case.

MLP networks drastically outperformed both Prophet and the ARIMA model in both markets based on correlation coefficient based backtesting. Naturally, this form of backtesting only measures the general consistency, which helps to decide whether to buy or sell a stock rather than a precise prediction of the entire future behavior of a stock. Hence, it cannot certainly be said that MLP networks perform better forecasting in the stock market compared to ARIMA and Prophet. Therefore, our results should be considered only within the correlation coefficient aspect. Figures 13-15 show the resulting histograms for test durations of 1 month, 3 months, and 6 months respectively. The prediction accuracy seems to be increasing from 1 month to 6 months of test duration, with a great portion of the correlation coefficients residing in the $[0.7, 1]$ range.

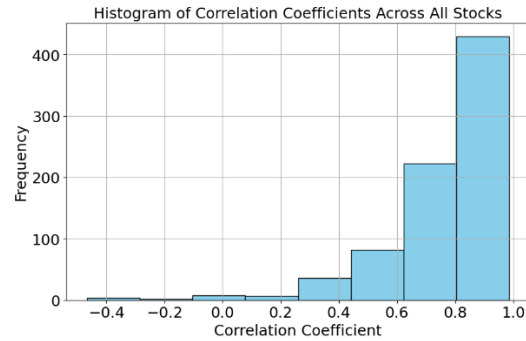


Fig. 15. Performance backtesting of MLP networks based on a testing data window of 1 month for 100 randomly selected corporate stock data from Yahoo Finance.

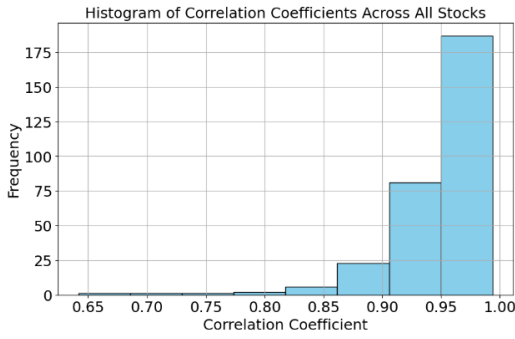


Fig. 16. Performance backtesting of MLP networks based on a testing data window of 3 months for 100 randomly selected corporate stock data from Yahoo Finance.

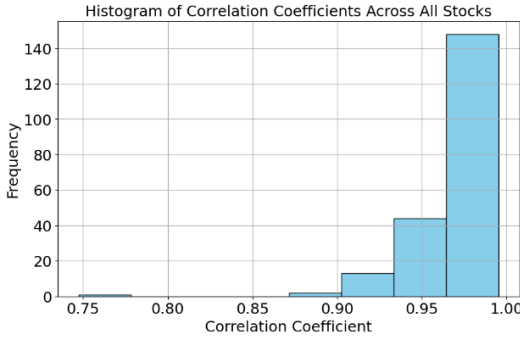


Fig. 17. Performance backtesting of MLP networks based on a testing data window of 6 months for 100 randomly selected corporate stock data from Yahoo Finance.

IV. DISCUSSION

In the comparative analysis, Prophet's performance varied significantly between the Turkish BIST30 and the broader dataset of 100 randomly selected stocks from Yahoo Finance. This discrepancy may be attributed to the nature of the datasets. The Turkish BIST30, which is a more homogeneous and possibly more predictable market, might have allowed Prophet to leverage its capabilities in handling seasonality and trend changes. In contrast, the dataset from Yahoo Finance encompassed a wide array of stocks with differing characteristics and volatilities. This diversity introduces complexities such as varying degrees of noise and non-stationarity, which can challenge Prophet's model assumptions and affect its overall performance.

In the observed performance differences between Prophet, ARIMA, and MLP networks, several key factors emerge. Prophet's superior performance over ARIMA can be attributed to its ability to handle the irregularities typical in financial time series data, such as missing values, outliers, and abrupt changes in trend (changepoints). Unlike ARIMA, which relies on traditional time series models that assume linearity and stable seasonality, Prophet introduces flexibility by modeling non-linear trends divided into daily, weekly, and yearly seasonality components, making it more suitable for adjusting to complex patterns observed in stock price movements.

However, despite these advantages, Prophet was outperformed by MLP networks. This outcome may be

attributed to the intrinsic capabilities of MLPs in managing and learning from large datasets with complex, non-linear interdependencies between variables. MLP networks, with their deep learning architecture, are designed to identify and exploit intricate patterns in the data that simpler models like Prophet and linear-based models like ARIMA may miss.

While Prophet offers a significant improvement over traditional models like ARIMA in handling non-linearities and seasonality, its simplicity, which is one of its strengths, might also limit its performance in scenarios where data complexity demands more sophisticated analytical capabilities, such as those provided by MLP networks. Therefore, the choice of model should consider the specific characteristics of the dataset and the complexity of the dynamics it contains.

The datasets that are used in this study contain hourly data, which means the time difference between two subsequent samples of a stock signal is 1 hour. The random selection of 100 stocks from Yahoo Finance did not yield significant changes in the results for different trials. Hence, the name-by-name statement of these stocks is not very relevant for the final results. The superiority of Prophet over ARIMA has also been suggested by other studies [1,2,5,6,17], which is in line with our observations. Likewise, one previous study has suggested the superiority of neural network based models over the plain Prophet model [9]. The comparative study in [18] has also shown that the performance of deep learning models is greater than those of ARIMA, especially in complex environments like stock market, supporting the findings of this study. Nevertheless, to our knowledge, our study is the first that makes a detailed quantitative comparison (see Table 1) between MLP networks (the simplest deep learning model), Prophet, and ARIMA.

TABLE I. COMPARISON OF MEAN CORRELATION COEFFICIENTS FOR EACH METHOD BASED ON THE DATA EXTRACTED FROM YAHOO FINANCE

Time Frame	ARIMA	Prophet	MLP Networks
1 Month	$r_{mean}=0.10$	$r_{mean}=0.09$	$r_{mean}=0.81$
3 Months	$r_{mean}=-0.05$	$r_{mean}=0.11$	$r_{mean}=0.96$
6 Months	$r_{mean}=-0.03$	$r_{mean}=-0.05$	$r_{mean}=0.96$

CONCLUSION

Our study revealed that Prophet outperformed ARIMA in correlation-coefficient-based backtesting, indicating its superior capability in capturing non-linear trends and managing irregularities inherent in financial time series data. However, Prophet did not match the performance of Multilayer Perceptron (MLP) networks, which demonstrated robust success across both the Turkish BIST30 market and the diverse environments of 100 randomly selected stocks from Yahoo Finance. While Prophet showed reasonable forecasting performance in the Turkish market, its performance was less consistent across the broader dataset that is randomly acquired from Yahoo Finance, indicating MLP's superior adaptability and accuracy in handling varied and complex data scenarios.

REFERENCES

- [1] B. Jin, S. Gao, and Z. Tao, "Arima and Facebook Prophet model in google stock price prediction," *Proceedings of Business and Economic Studies*, vol. 5, no. 5, pp. 60–66, Oct. 2022.
- [2] M. Daraghme, A. Agarwal, R. Manzano, and M. Zaman, "Time Series forecasting using Facebook Prophet for Cloud Resource Management," 2021 IEEE International Conference on Communications Workshops (ICC Workshops), Jun. 2021.
- [3] R. Gupta, A. K. Yadav, S. Jha, and P. K. Pathak, "Time series forecasting of solar power generation using Facebook prophet and XG boost," 2022 IEEE Delhi Section Conference (DELCON), Feb. 2022.
- [4] Q. Huang, "Forecasting stock prices using multi-macroeconomic regressors based on the facebook prophet model," *BCP Business & Management*, vol. 25, pp. 231–242, Aug. 2022.
- [5] F. V. Ferdinand, T. H. Santoso, and K. V. Saputra, "Performance comparison between Facebook Prophet and Sarima on Indonesian stock," 2023 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM), Dec. 2023.
- [6] A. Garlapati, D. R. Krishna, K. Garlapati, N. m. Srikara Yaswanth, U. Rahul and G. Narayanan, "Stock Price Prediction Using Facebook Prophet and Arima Models," 2021 6th International Conference for Convergence in Technology (I2CT), Maharashtra, India, 2021, pp. 1-7.
- [7] Jingyi Shen and M. Omair Shafiq, "Short term stock market price trend prediction using a comprehensive deep learning system", *Journal of Big data* 7, vol. 66, 2020.
- [8] K. Albeladi, B. Zafar, and A. Mueen, A novel deep-learning based approach for Time Series Forecasting using Sarima, Neural Prophet and FB Prophet, Nov. 2023.
- [9] N. Afarini and D. Hindarto, "Forecasting airline passenger growth: Comparative study LSTM vs Prophet vs neural prophet," *Sinkron*, vol. 9, no. 1, pp. 505–513, Jan. 2024.
- [10] A. Yurttabir and I. K. Sen, "Financial Performance Forecasting Using prophet model: An application in BIST manufacturing sector," *Pressacademia*, vol. 14, no. 1, pp. 130–132, Dec. 2021.
- [11] N. Saeed, S. Nguyen, K. Cullinane, V. Gekara, and P. Chhetri, "Forecasting container freight rates using the prophet forecasting method," *Transport Policy*, vol. 133, pp. 86–107, Mar. 2023.
- [12] J. Kolari and I. P. Sanz, "Forecasting bank capital ratios using the Prophet model by Facebook," *Journal of Finance Issues*, vol. 20, no. 3, pp. 40–64, Dec. 2022.
- [13] S. Kaninde, M. Mahajan, A. Janghale, and B. Joshi, "Stock price prediction using Facebook Prophet," *ITM Web of Conferences*, vol. 44, p. 03060, 2022.
- [14] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [15] S. Burke et al., "Application of Prophet models in forecasting electricity demand," *Journal of Electrical Systems*, vol. 11, no. 2, pp. 209–223, 2020.
- [16] L. Smith et al., "Using Facebook's Prophet Forecasting Algorithm to Predict Infectious Disease Spread," *Journal of Public Health Research*, vol. 8, no. 3, pp. 1642–1649, 2019.
- [17] S. R. Jammalamadaka and A. Sengupta, "Time Series Forecasting in Financial Markets: An Application of Prophet and ARIMA," *Journal of Risk and Financial Management*, vol. 14, no. 4, Art. no. 162, 2021.
- [18] Z. Gao, "Stock price prediction with Arima and Deep Learning Models," 2021 IEEE 6th International Conference on Big Data Analytics (ICBDA), Mar. 2021.
- [19] H. Ji, "Stock price prediction based on SVM, LSTM, Arima," *BCP Business & Management*, vol. 35, pp. 267–272, Dec. 2022.
- [20] A. Aşırım, Ö. E. Aşırım, and M. A. Salepçioğlu, "Analysis of local system behavior in the foreign exchange-market using neural networks and Monte-Carlo method for prediction and risk assessment," *SN Appl. Sci.*, vol. 5, no. 78, 2023.